

SIMATS ENGINEERING
CO2-ASSESSMENT TOOL 2

COURSE NAME: Design and Analysis of Algorithms

COURSE CODE: CSA0613

Name : C.Himaja

Reg.No : 192373035

Q11. A banking system stores daily transaction amounts in a list where new entries are appended randomly, causing disorder. The system needs to sort data periodically for reporting.

- a) Analyze how Selection Sort and Bubble Sort would work in this scenario.**
- b) Compare the number of comparisons and swaps required by both methods.**
- c) Recommend the most suitable sorting technique with proper justification.**

Title

Comparative Analysis of Selection Sort and Bubble Sort for Banking Transaction Records

Aim

To analyze and compare the performance of Selection Sort and Bubble Sort for sorting randomly appended banking transaction amounts and recommend the most suitable algorithm.

Problem Statement

A banking system stores daily transaction amounts in random order because new transactions are continuously appended. Before generating daily reports, the transactions must be sorted efficiently.

Objectives

- Study Selection Sort.
- Study Bubble Sort.
- Compare comparisons and swaps.
- Recommend the better algorithm.

Algorithm

Selection Sort

1. Find the smallest element.
2. Swap it with the first unsorted element.
3. Repeat until the array is sorted.

Bubble Sort

1. Compare adjacent elements.
2. Swap if they are in the wrong order.
3. Repeat until no more swaps are needed.

Analysis

a) Working of Both Algorithms

Selection Sort

- Finds the minimum transaction amount.
- Places it at the beginning.
- Repeats for the remaining elements.
- Makes very few swaps.

Example

4500 1200 3800 2500 5100 1900

↓

1200 4500 3800 2500 5100 1900

↓

1200 1900 3800 2500 5100 4500

↓

1200 1900 2500 3800 4500 5100

Bubble Sort

- Compares adjacent transactions.
- Larger values gradually move to the end.
- Repeatedly passes through the array.

Example

4500 1200 3800 2500 5100 1900

↓

1200 3800 2500 4500 1900 5100

↓

1200 2500 3800 1900 4500 5100

↓

1200 1900 2500 3800 4500 5100

b) Comparison of Comparisons and Swaps

For n elements

Selection Sort

Comparisons

$$n(n-1)/2$$

Swaps

$$\text{Maximum} = n-1$$

Bubble Sort

Comparisons

$$n(n-1)/2$$

Swaps

Depends on disorder.

Worst Case

$$n(n-1)/2$$

Comparison Table

Feature	Selection Sort	Bubble Sort
Comparisons	$n(n-1)/2$	$n(n-1)/2$
Swaps	Very Few	Many
Stability	Not Stable	Stable
Memory	O(1)	O(1)

Feature	Selection Sort	Bubble Sort
----------------	-----------------------	--------------------

Performance	Better	Slower
-------------	--------	--------

c) Recommendation

Selection Sort is more suitable for this banking scenario because:

- Banking records require fewer write operations.
- Selection Sort performs significantly fewer swaps.
- Less data movement improves efficiency.
- Suitable for periodic sorting of small to medium datasets.

Bubble Sort performs unnecessary swaps, making it slower for randomly ordered banking transactions.

Complexity Analysis

Algorithm	Time	Space
------------------	-------------	--------------

Selection Sort	$O(n^2)$	$O(1)$
----------------	----------	--------

Bubble Sort	$O(n^2)$	$O(1)$
-------------	----------	--------

Advantages

Selection Sort

- Fewer swaps
- Easy to implement
- Good for small datasets

Bubble Sort

- Simple algorithm
- Stable sorting
- Easy to understand

Disadvantages

Selection Sort

- Not stable
- Slow for large datasets

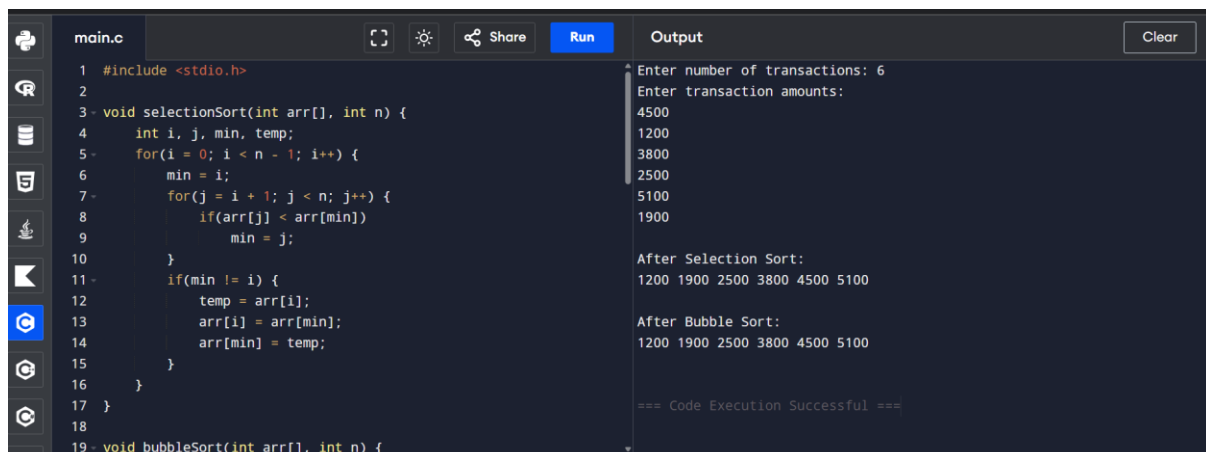
Bubble Sort

- Performs many swaps
- Inefficient on large datasets

Conclusion

Both algorithms correctly sort banking transaction records. However, Selection Sort is preferred because it performs the same number of comparisons as Bubble Sort but requires far fewer swaps. This reduces unnecessary data movement and makes it more efficient for periodically sorting banking transaction data.

Screenshots to Include in the Report



The screenshot shows a C program in a code editor with the following code:

```
main.c
1 #include <stdio.h>
2
3 void selectionSort(int arr[], int n) {
4     int i, j, min, temp;
5     for(i = 0; i < n - 1; i++) {
6         min = i;
7         for(j = i + 1; j < n; j++) {
8             if(arr[j] < arr[min])
9                 min = j;
10        }
11        if(min != i) {
12            temp = arr[i];
13            arr[i] = arr[min];
14            arr[min] = temp;
15        }
16    }
17 }
18
19 void bubbleSort(int arr[], int n) {
```

The output window shows the following text:

```
Enter number of transactions: 6
Enter transaction amounts:
4500
1200
3800
2500
5100
1900

After Selection Sort:
1200 1900 2500 3800 4500 5100

After Bubble Sort:
1200 1900 2500 3800 4500 5100

=== Code Execution Successful ===
```

main.c

Run

Share

Settings

Fullscreen

20

int i, j, temp;

21

for(i = 0; i < n - 1; i++) {

22

for(j = 0; j < n - i - 1; j++) {

23

if(arr[j] > arr[j + 1]) {

24

temp = arr[j];

25

arr[j] = arr[j + 1];

26

arr[j + 1] = temp;

27

}

28

}

29

}

30

}

31

32

void printArray(int arr[], int n) {

33

int i;

34

for(i = 0; i < n; i++)

35

printf("%d ", arr[i]);

36

printf("\n");

37

}

Output

Clear

Enter number of transactions: 6

Enter transaction amounts:

4500

1200

3800

2500

5100

1900

After Selection Sort:

1200 1900 2500 3800 4500 5100

After Bubble Sort:

1200 1900 2500 3800 4500 5100

=== Code Execution Successful ===

main.c

Run

Share

Settings

Fullscreen

38

39

int main() {

40

int n, i;

41

42

printf("Enter number of transactions: ");

43

scanf("%d", &n);

44

45

int a[n], b[n];

46

47

printf("Enter transaction amounts:\n");

48

for(i = 0; i < n; i++) {

49

scanf("%d", &a[i]);

50

b[i] = a[i];

51

}

52

53

selectionSort(a, n);

54

printf("\nAfter Selection Sort:\n");

55

printArray(a, n);

56

}

Output

Clear

Enter number of transactions: 6

Enter transaction amounts:

4500

1200

3800

2500

5100

1900

After Selection Sort:

1200 1900 2500 3800 4500 5100

After Bubble Sort:

1200 1900 2500 3800 4500 5100

=== Code Execution Successful ===

main.c

Run

Share

Settings

Fullscreen

44

45

int a[n], b[n];

46

47

printf("Enter transaction amounts:\n");

48

for(i = 0; i < n; i++) {

49

scanf("%d", &a[i]);

50

b[i] = a[i];

51

}

52

53

selectionSort(a, n);

54

printf("\nAfter Selection Sort:\n");

55

printArray(a, n);

56

57

bubbleSort(b, n);

58

printf("\nAfter Bubble Sort:\n");

59

printArray(b, n);

60

61

return 0;

62

}

Output

Clear

Enter number of transactions: 6

Enter transaction amounts:

4500

1200

3800

2500

5100

1900

After Selection Sort:

1200 1900 2500 3800 4500 5100

After Bubble Sort:

1200 1900 2500 3800 4500 5100

=== Code Execution Successful ===